

图 13-6 判定树描述的某公司折扣策略

13.3 结构化设计

结构化设计（Structured Design, SD）是一种面向数据流的方法，它以 SRS 和 SA 阶段所产生的数据流图和数据字典等文档为基础，是一个自顶向下、逐步求精和模块化的过程。SD 方法的基本思想是将软件设计成由相对独立且具有单一功能的模块组成的结构。软件设计可分为概要设计和详细设计两个阶段，其中概要设计的主要任务是确定软件系统的结构，对系统进行模块划分，确定每个模块的功能、接口和模块之间的调用关系；详细设计的主要任务是为每个模块设计实现的细节。

13.3.1 模块结构

系统是一个整体，它具有整体性的目标和功能，但这些目标和功能的实现又是由相互联系各个组成部分共同工作的结果。人们在解决复杂问题时使用的一个很重要的原则，就是将它分解成多个小问题分别处理。在处理过程中，需要根据系统总体要求，协调各业务部门的关系。在 SD 中，这种功能分解就是将系统划分为模块，模块是组成系统的基本单位，它的特点是可以自由组合、分解和变换，系统中任何一个处理功能都可以看成一个模块。

1. 信息隐蔽与抽象

信息隐蔽原则要求采用封装技术，将程序模块的实现细节（过程或数据）隐藏起来，对于不需要这些信息的其他模块来说是不能访问的，使模块接口尽量简单。按照信息隐蔽原则，系统中的模块应设计成“黑盒”，模块外部只能使用模块接口说明中给出的信息，例如，操作和数据类型等。模块之间相对独立，这样既易于实现，也易于理解和维护。

抽象原则要求抽取事物最基本的特性和行为，忽略非本质的细节，采用分层次抽象的方式可以控制软件开发过程的复杂性，有利于软件的可理解性和开发过程的管理。通常，抽象层次包括过程抽象、数据抽象和控制抽象。

2. 模块化

在 SD 方法中，模块是实现功能的基本单位，它一般具有功能、逻辑和状态三个基本属性，其中功能是指该模块“做什么”，逻辑用于描述模块内部“怎么做”，状态是指该模块使用时的环境和条件。在描述一个模块时，必须按模块的外部特性与内部特性分别描述。模块的外部特

性是指模块的模块名、参数表和给程序乃至整个系统造成的影响；模块的内部特性则是指完成其功能的程序代码和仅供该模块内部使用的数据。对于模块的外部环境（例如，需要调用这个模块的上级模块）来说，只需要了解这个模块的外部特性就足够了，不必了解它的内部特性。在软件设计阶段通常是先确定模块的外部特性，然后再确定它的内部特性。

在SD方法中，系统由多个逻辑上相对独立的模块组成，在模块划分时需要遵循如下原则：

（1）模块的大小要适中。系统分解时需要考虑模块的规模，过大的模块可能导致系统分解不充分，其内部可能包括不同类型的功能，需要进一步划分，尽量使各个模块的功能单一；过小的模块将导致系统的复杂度增加，模块之间的调用过于频繁，反而降低了模块的独立性。一般来说，当一个模块的大小使其实现代码约1~2页纸，或者其实现代码行数约50~200行时，这种规模的模块易于实现和维护。

（2）模块的扇入和扇出要合理。一个模块的扇出是指该模块直接调用的下级模块的个数。扇出大表示模块的复杂度高，需要控制和协调过多的下级模块。扇出过大一般是因为缺乏中间层次，应该适当增加中间层次的控制模块。扇出太小时可以把下级模块进一步分解成若干个子功能模块，或者合并到它的上级模块中去。一个模块的扇入是指直接调用该模块的上级模块的个数。扇入大表示模块的复用程度高。设计良好的软件结构通常顶层扇出比较大，中间扇出较少，底层模块则扇入大。一般来说，系统的平均扇入和扇出系数为3或4，不应该超过7，否则会增大出错的概率。

（3）深度和宽度适当。深度表示软件结构中模块的层数，如果层数过多，则应考虑有些模块设计是否过于简单，看能否适当合并。宽度是指软件结构中同一个层次上的模块总数的最大值，一般说来，宽度越大系统越复杂。对宽度影响最大的因素是模块的扇出。在系统设计时，需要权衡系统的深度和宽度，尽量降低系统的复杂性，减少实施过程的难度，提高开发和维护的效率。

3. 耦合

耦合表示模块之间联系的程度。紧密耦合表示模块之间联系非常强；松散耦合表示模块之间联系比较弱；非耦合则表示模块之间无任何联系，是完全独立的。模块的耦合类型通常分为7种，根据耦合度从低到高排序如表13-3所示。

表 13-3 模块的耦合类型

耦合类型	描 述
非直接耦合	两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的
数据耦合	一组模块借助参数表传递简单数据
标记耦合	一组模块通过参数表传递记录信息（数据结构）
控制耦合	模块之间传递的信息中包含用于控制模块内部逻辑的信息
外部耦合	一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息
公共耦合	多个模块都访问同一个公共数据环境，公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等
内容耦合	包括：一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口

对于模块之间耦合的强度，主要依赖于一个模块对另一个模块的调用、一个模块向另一个模块传递的数据量、一个模块施加到另一个模块的控制的多少，以及模块之间接口的复杂程度。

4. 内聚

内聚表示模块内部各成分之间的联系程度，是从功能角度来度量模块内的联系，一个好的内聚模块应当恰好做目标单一的一件事情。模块的内聚类型通常也可以分为7种，根据内聚度从高到低排序如表13-4所示。

表 13-4 模块的内聚类型

内聚类型	描 述
功能内聚	完成一个单一功能，各个部分协同工作，缺一不可
顺序内聚	处理元素相关，而且必须顺序执行
通信内聚	所有处理元素集中在一个数据结构的区域上
过程内聚	处理元素相关，而且必须按特定的次序执行
瞬时内聚（时间内聚）	所包含的任务必须在同一时间间隔内执行
逻辑内聚	完成逻辑上相关的一组任务
偶然内聚（巧合内聚）	完成一组没有关系或松散关系的任务

一般说来，系统中各模块的内聚越高，则模块间的耦合就越低，但这种关系并不是绝对的。耦合低使得模块间尽可能相对独立，从而各模块可以单独开发和维护；内聚高使得模块的可理解性和维护性大大增强。因此，在模块的分解中应尽量减少模块的耦合，力求增加模块的内聚，遵循“高内聚、低耦合”的设计原则。

5. 模块类型

在系统结构图（Structure Chart, SC）中不能再分解的底层模块称为原子模块。如果一个系统的全部实际加工（数据计算或处理）都由底层的原子模块来完成，而其他所有非原子模块仅仅执行控制或协调功能，这样的系统就是完全因子分解的系统。如果 SC 是完全因子分解的，就是最好的系统。一般而言，在 SC 中存在4种类型的模块，如图13-7所示。

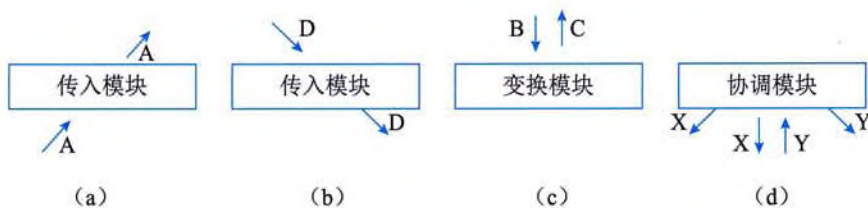


图 13-7 SC 的4种模块类型

(1) 传入模块。传入模块从下属模块中获取数据，经过某些处理，再将其传送给上级模块，如图13-7(a)所示。

(2) 传出模块。传出模块从上级模块中获取数据，进行某些处理，再将其传送给下属模块，如图13-7(b)所示。